คู่มือปฏิบัติการ Apache Airflow Mini Lab (30 นาที): การสร้างและจัดการ ML Pipeline

Ref: https://airflow.apache.org **และ** https://airflow.apache.org/docs/apache-airflow/stable/index.html

1. บทนำและความสำคัญของ Apache Airflow ในงานข้อมูล

ในฐานะวิศวกรข้อมูล สิ่งสำคัญที่สุดไม่ใช่เพียงแค่การเขียนสคริปต์ให้ทำงานได้ แต่คือการบริหารจัดการ "วงจรชีวิต" ของข้อมูลอย่างเป็นระบบ **Apache Airflow** คือแพลตฟอร์ม Open-source มาตรฐานระดับโลกสำหรับ Workflow Orchestration ที่ช่วยให้เราแยกส่วน **Processing Logic** (ตัวงาน) ออกจาก **Orchestration Logic** (การจัดการลำดับและเวลา) อย่างเด็ดขาด

ทำไมองค์กรระดับ Enterprise ต้องใช้ Airflow?

การรัน Python Script เพียงไฟล์เดียว (Single Script) อาจใช้ได้กับงานขนาดเล็ก แต่เมื่อระบบซับซ้อนขึ้น Airflow จะเข้ามาแก้ปัญหาเชิงยุทธศาสตร์ดังนี้:

- **Scalability:** สถาปัตยกรรมแบบ Modular และการใช้ Message Queue ช่วยให้สามารถกระจายงานไปยัง Worker จำนวนมากได้พร้อมกัน รองรับการขยายตัวอย่างไร้ขีดจำกัด
- **Dynamic Workflows:** Pipeline ถูกนิยามด้วยโค้ด Python ทำให้เราสร้างลำดับงานที่ยืดหยุ่นและปรับเปลี่ยนตามเงื่อนไข (Dynamic generation) ได้ทันที
- **Observability & Monitoring:** เปลี่ยนระบบ "Black-box" ให้โปร่งใสด้วย Web UI ที่แสดงสถานะงานแบบ Real-time พร้อมระบบ Alerting เมื่อเกิดปัญหา

ความเข้าใจในโครงสร้างเหล่านี้จะเปลี่ยนวิธีการมองการส่งต่อข้อมูล (Data Pipeline) จากการเป็นแค่ "สคริปต์ที่รันต่อกัน" ไปสู่การเป็น "ระบบนิเวศข้อมูลที่ตรวจสอบได้"

---

2. โครงสร้างและแนวคิดหลัก (Core Concepts)

การออกแบบ Pipeline ที่มีประสิทธิภาพต้องวางรากฐานบนความเข้าใจในองค์ประกอบ 5 ประการ ซึ่งทำหน้าที่ประสานงานกันเพื่อให้ Workflow ทำงานได้อย่างราบรื่น

ตารางสรุปแนวคิดหลัก

แนวคิด	นิยาม	หน้าที่เชิงยุทธศาสตร์
DAG	Directed Acyclic Graph	"แผนที่นำทาง" ที่ระบุความสัมพันธ์และลำดับการไหลของงาน
Task	หน่วยย่อยของงาน	จุดที่เกิดการทำงานจริง (Execution point) ในแต่ละขั้นตอน
Operator	ตัวกำหนดประเภทงาน	ระบุ "วิธีการ" ทำงาน (เช่น BashOperator สำหรับคำสั่ง Shell)
Scheduler	ตัวจัดการตารางเวลา	"หัวใจ" ของระบบที่คอย Monitor DAG และสั่งรัน Task ทันทีที่เงื่อนไขครบ
Web UI	ส่วนติดต่อผู้ใช้งาน	ศูนย์กลางการบริหารจัดการ ตรวจสอบ Logs และ Debug งาน

ความสำคัญเชิงลึกของ "Acyclic" ใน DAG

DAG เปรียบเสมือนพิมพ์เขียวที่ต้องไม่มีวงจรซ้ำซ้อน (**Acyclic**) การที่กราฟไม่มีการวนกลับทำให้ Scheduler สามารถคำนวณ **Topological Sort** เพื่อหาลำดับการทำงานที่แน่นอนได้ ช่วยป้องกันการเกิด Deadlock และทำให้สถานะการทำงาน (State Management) เป็นไปอย่างชัดเจนและคาดเดาผลลัพธ์ได้ (Deterministic)

---

3. การจัดเตรียมสภาพแวดล้อมและโครงสร้างโครงการ

ในการปฏิบัติการนี้ เราจะใช้สภาพแวดล้อมแบบ **Docker Compose** เพื่อเลียนแบบสภาพแวดล้อมจริง (Production-like) ซึ่งช่วยให้มั่นใจว่า Pipeline จะทำงานได้เหมือนกันในทุกเครื่อง

ขั้นตอนการติดตั้งและเตรียมระบบ

1. **ดึงไฟล์ตั้งค่า:** ใช้คำสั่ง curl -LfO 'https://airflow.apache.org/docs/apache-airflow/stable/docker-compose.yaml'
2. **สร้างโฟลเดอร์โครงการ:** mkdir -p ./dags ./logs ./plugins *Note: การสร้างโฟลเดอร์เหล่านี้เป็นการทำ Volume Mapping เพื่อให้โค้ดที่เราเขียนในเครื่องสะท้อนเข้าไปใน Container ทันที*

3. **ตั้งค่าสิทธิ์และเริ่มต้น:** กำหนด AIRFLOW_UID ในไฟล์ .env จากนั้นรัน docker compose up airflow-init เพื่อเตรียมฐานข้อมูล และรัน docker compose up เพื่อเริ่มบริการ
(* ในเครื่องต้องติดตั้ง Docker ก่อน โดยไปที่ https://www.docker.com/products/docker-desktop/ และติดตั้ง version ที่ถูกต้อง
** ถ้า AIRFLOW_UID ในไฟล์ .env ไม่ได้ เช่น สร้างไม่ได้ หรือมองไม่เห็น hidden file ให้ถาม ChatGPT หรือ Gemini ว่าทำอย่างไร โดย upload ไฟล์ Tutorial นี้ขึ้นไปก่อนค่อยถาม)

รายการตรวจสอบความพร้อม (Pre-flight Checklist)

- [ ] บริการ Docker ทุกตัวอยู่ในสถานะ "Healthy" หรือ "Running"
- [ ] มีโฟลเดอร์ dags/ สำหรับวางไฟล์ Python เรียบร้อยแล้ว
- [ ] สามารถเข้าถึง Web UI ได้ที่ http://localhost:8080 (Username/PW: airflow)

---

4. ปฏิบัติการ: การสร้าง Simple ML Pipeline (Step-by-Step)

วัตถุประสงค์คือการสร้าง Pipeline ตามวงจรชีวิต ML: start ⟶ preprocess ⟶ train ⟶ evaluate เพื่อให้มั่นใจว่าการเทรนโมเดลจะเกิดขึ้นหลังจากข้อมูลพร้อมแล้วเท่านั้น

Step 1: สร้างไฟล์ DAG (ml_pipeline.py)

สร้างไฟล์ชื่อ “**ml_pipeline.py**” ใน dags/ และใส่ code ดังต่อไปนี้แล้ว save
(สังเกตการใช้ catchup=False เพื่อป้องกันการรันงานย้อนหลังที่ไม่จำเป็น)

```
from datetime import datetime, timedelta
from airflow.sdk import DAG
from airflow.providers.standard.operators.bash import BashOperator

default_args = {
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}
```

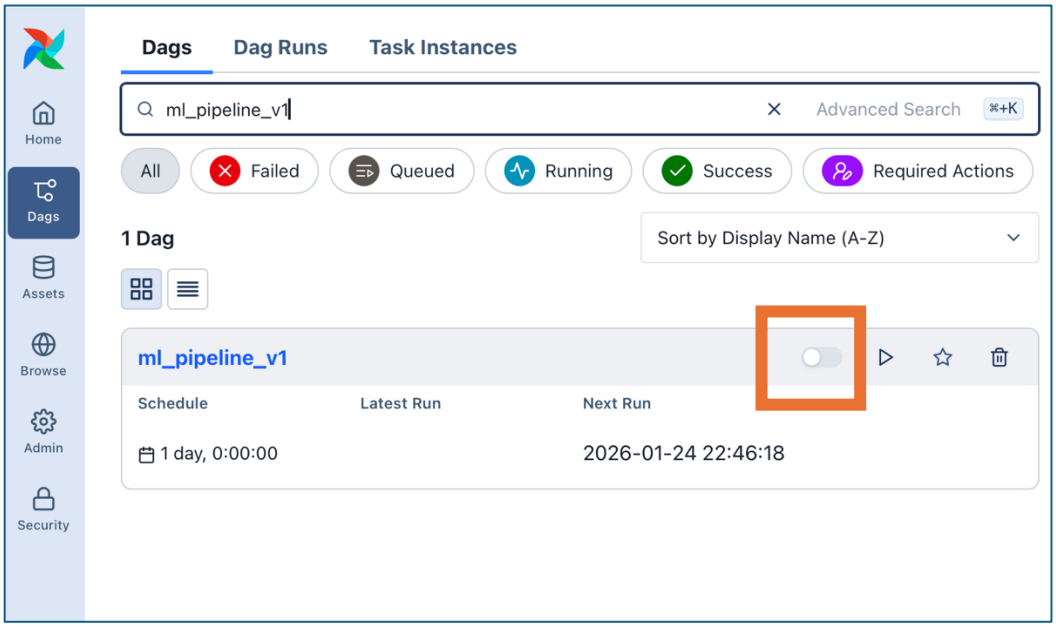
```
with DAG(
    "ml_pipeline_v1",
    default_args=default_args,
    schedule=timedelta(days=1),
    start_date=datetime(2021, 1, 1),
    catchup=False, # ห้ามใช้ catch_date
) as dag:

    t1 = BashOperator(task_id="start", bash_command="echo 'Starting ML Pipeline'")
    t2 = BashOperator(task_id="preprocess", bash_command="echo 'Preprocessing data...'")
    t3 = BashOperator(task_id="train", bash_command="echo 'Training model...'")
    t4 = BashOperator(task_id="evaluate", bash_command="echo 'Evaluating results...'")

    t1 >> t2 >> t3 >> t4  # กำหนด Dependency แบบ Chaining
```

Step 2: Trigger งานและติดตามผล

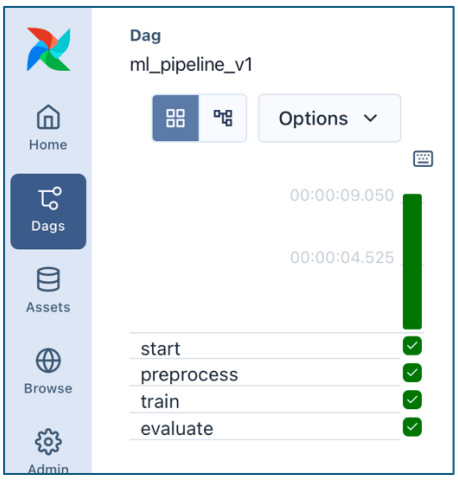
ที่หน้า UI ให้เลือก Dags และ search “ml_pipeline_v1” (ตามในรูป)



เปิด Toggle "On" ที่ ml_pipeline_v1 ในหน้า UI:

- **Graph View:** ตรวจสอบ "ทิศทาง" ของงานว่าตรงตามที่ออกแบบไว้หรือไม่
- **Grid View:** ใช้ดูภาพรวมความสำเร็จ (Success Rate) ในแต่ละรอบเวลา

(ตัวอย่าง)



Step 3: การจำลองความล้มเหลว (Debugging Exercise)

เราจะลองเปลี่ยน bash_command ใน Task train เป็น exit 1 เพื่อจำลอง Error:

1) ให้เปิดไฟล์ “dags/ml_pipeline_v1.py”

❌ ก่อนแก้ (ปกติ) >>

```
t3 = BashOperator(
    task_id="train",
    bash_command="echo 'Training model...'"
)
```

✅ แก้เป็น (จำลอง Error)

```
t3 = BashOperator(
    task_id="train",
    bash_command="exit 1"
)
```

#หมายเหตุ: exit 1 = คำสั่ง Linux ที่บอกว่า “จบโปรแกรมด้วยสถานะผิดพลาด (non-zero exit code)”

2) ทำการ save ไฟล์แล้วรอ ~10–30 วินาที (ไม่ต้อง restart container)

3) Trigger DAG ใหม่จาก Web UI

1. ไปที่ http://localhost:8080
2. เปิด DAG ml_pipeline_v1
3. กดปุ่ม ▶️ Trigger DAG

4) สังเกตผลใน Graph View แล้วจะเห็นว่า:

- start ⟶ ✅ success
- preprocess ⟶ ✅ success
- train ⟶ up_for_retry หรือ ❌ failed (สีแดง)
- evaluate ⟶ ⛔ ไม่ถูกรัน (Skipped / None) หรือ ❌ upstream_failed (สีแดง) หรือ no_status

5) ดู Log ของ Task ที่ล้มเหลว

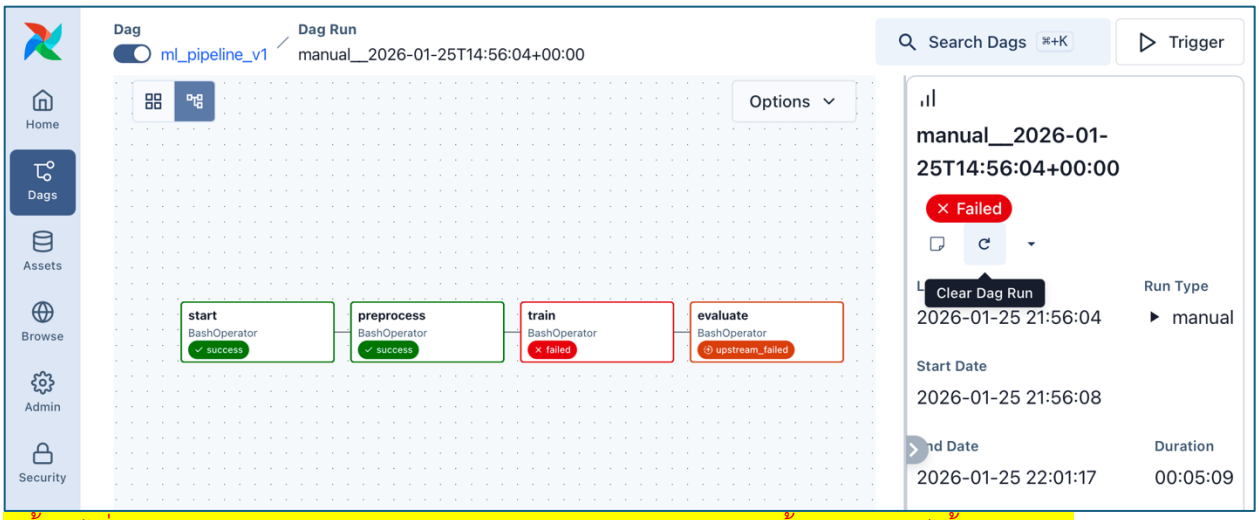
1. คลิกที่กล่อง task train
2. เลือก Logs
3. นิสิตควรเห็นข้อความประมาณ: Command exited with return code 1

6) แก้โค้ดกลับ

```
t3 = BashOperator(
    task_id="train",
    bash_command="echo 'Training model...'"
)
```

Clear เฉพาะ Task ที่พัง

- คลิก task train
- กด Clear
- ** เลือก ✅ Clear this task only **



(*ถ้าหาไม่เจอจริงๆ ลองถาม ChatGPT หรือ Gemini ดู เพราะจริงๆ แล้วสามารถกดได้หลายทาง)

---

5. การวิเคราะห์เชิงวิพากษ์และคำถามชวนคิด (Reflection & Analysis)

ความเชื่อมั่นในระบบ (Reliability)

Airflow เหนือกว่าการรันสคริปต์ทั่วไปตรงที่มันมีระบบ **Dependency Management** หาก Task preprocess ล้มเหลว Task trainและ evaluate (Downstream) จะไม่ถูกรันเด็ดขาด ช่วยป้องกันการใช้ข้อมูลที่ผิดพลาดมาสร้างโมเดล

Architect's Note: Idempotency & Atomicity

- **Idempotency:** การรัน Task เดิมซ้ำด้วย Input เดิมต้องได้ผลลัพธ์เดิมเสมอ ไม่สร้างผลกระทบข้างเคียงซ้ำซ้อน
- **Atomicity:** แต่ละ Task ควรทำงานเพียงอย่างเดียวที่จบในตัว (Atomic operation) เพื่อให้การ Retry และการ Monitor มีประสิทธิภาพสูงสุด

So What? Layer: สำหรับ Data Engineer นั้น Airflow ไม่ได้เป็นแค่ตัวรันโค้ด แต่เป็นเครื่องมือสร้าง **Observability** ที่ทำให้เราเห็นจุดตายของ Data Pipeline ได้ทันท่วงทีก่อนที่ข้อมูลผิดๆ จะถึงมือ User

---

6. สรุปและแนวทางการต่อยอด (Key Takeaways & Next Steps)

Apache Airflow คือหัวใจของ **Modern MLOps stack** ที่ช่วยเปลี่ยนสคริปต์ที่กระจัดกระจายให้กลายเป็นระบบงานที่พร้อมสำหรับ Production

หัวข้อสำหรับการเรียนรู้ขั้นต่อไป

- **TaskFlow API:** ศึกษาการใช้ Decorator (@task) เพื่อเขียน DAG ในรูปแบบ "Pythonic" ที่สะอาดตา และจัดการ XComs อัตโนมัติ
- **ObjectStoragePath:** ใช้ Abstraction ใหม่ใน Airflow 3.x สำหรับจัดการ Cloud Storage (S3, GCS) แบบ Cloud-agnostic (ทำงานคล้าย pathlib.Path แต่ใช้กับ Bucket)
- **HITLOperator (Human-in-the-loop):** ฟีเจอร์ใหม่ในเวอร์ชัน 3.1 สำหรับขั้นตอนที่ต้องมีการอนุมัติ (Approval) จากมนุษย์ก่อนทำงานต่อ

Best Practices ที่ต้องยึดถือ

1. **Jinja Templating:** ใช้ {{ ds }} หรือตัวแปรเวลาเพื่อให้ DAG มีความยืดหยุ่น (Dynamic)
2. **Documentation:** ใส่คำอธิบายใน doc_md ของ DAG/Task เสมอ เพื่อลดภาระในการส่งต่องาน
3. **Automatic Retries:** ตั้งค่า retries เสมอเพื่อจัดการกับ Transient failures (เช่น ปัญหา Network ชั่วคราว)

Key Takeaway: "ความสำเร็จของ ML Pipeline ไม่ได้วัดที่ความแม่นยำของโมเดลเพียงอย่างเดียว แต่วัดที่ความสามารถในการรันซ้ำได้ (Reproducibility) และการตรวจพบปัญหาได้ทันที (Fast Failure Detection)"